

**LAPORAN TUGAS
SISTEM BASIS DATA**

Sistem Manajemen Stok Barang Elektronik



Disusun Oleh:

Kelompok 4

Floencia Irena Nifili Gulo	(2510631250033)
Muhammad Dafa Anggara Putra	(2510631250012)
Gathan Abramovic	(2510631250063)
Shinta Aprilia	(2510631250048)

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
2026**

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga proyek akhir mata kuliah Algoritma dan Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan Project UAS yang telah dikerjakan oleh Kelompok 4. Dalam proyek ini, kami membangun sebuah Sistem Manajemen Stok Toko Elektronik berbasis konsol yang menerapkan berbagai konsep algoritma dan struktur data yang telah dipelajari selama perkuliahan.

Kami menyadari bahwa proyek ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan kedepannya. Kami juga mengucapkan terima kasih kepada dosen pengampu mata kuliah Algoritma dan Struktur Data, serta seluruh pihak yang telah memberikan dukungan dalam penyelesaian proyek ini.

Semoga laporan dan sistem yang kami kembangkan ini dapat memberikan manfaat, khususnya sebagai referensi penerapan algoritma dan struktur data dalam membangun aplikasi yang fungsional.

Karawang, 28 Mei 2026

Kelompok 4

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
BAB II ANALISIS KEBUTUHAN.....	3
2.1 Analisis Kebutuhan Sistem.....	3
2.2 Kebutuhan Fungsional.....	3
2.3 Struktur Data yang Digunakan.....	3
2.4 Algoritma yang Digunakan.....	4
2.5 Analisis Kebutuhan Non-Fungsional.....	4
2.6 Konsep Pemrograman yang Digunakan.....	4
BAB III IMPLEMENTASI.....	5
3.1 Implementasi Kode dan Penjelasan.....	5
BAB IV ANALISIS KOMPLEKSITAS.....	27
4.1 Tabel Analisis Kompleksitas Algoritma.....	27
4.2 Penjelasan Analisis Kompleksitas Algoritma.....	28
BAB V PENUTUP.....	29
5.1 Kesimpulan.....	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong berbagai sektor usaha untuk mengadopsi sistem pengelolaan data yang lebih terstruktur dan efisien, termasuk di dalamnya usaha retail elektronik. Toko elektronik umumnya memiliki beragam jenis produk dengan jumlah stok yang terus berubah setiap harinya. Tanpa sistem pencatatan yang baik, pengelolaan stok barang rentan terhadap kesalahan pencatatan, kehilangan data, serta ketidakakuratan informasi yang dapat merugikan operasional toko.

Selama ini, pencatatan stok pada toko berskala kecil hingga menengah masih sering dilakukan secara manual, sehingga proses pencarian data, pembaruan stok, maupun pengurutan informasi produk membutuhkan waktu yang tidak sedikit dan berpotensi menimbulkan kesalahan manusia (human error). Kondisi ini menjadi dasar perlunya sebuah sistem yang mampu mengelola data stok secara cepat, terorganisir, dan dapat diandalkan.

Mata kuliah Algoritma dan Struktur Data memberikan fondasi ilmu yang relevan untuk menjawab permasalahan tersebut. Konsep-konsep seperti array, record, algoritma pengurutan (sorting), dan algoritma pencarian (searching) merupakan alat yang tepat untuk membangun sistem manajemen stok yang efisien. Oleh karena itu, Kelompok 4 mengembangkan Sistem Manajemen Stok Toko Elektronik berbasis konsol sebagai implementasi nyata dari konsep-konsep yang telah dipelajari, sekaligus membuktikan bahwa algoritma dan struktur data dapat diterapkan langsung dalam membangun aplikasi sederhana yang fungsional.

1.2 Tujuan

Tujuan dari pengembangan sistem ini adalah:

1. Membangun sistem manajemen stok toko elektronik berbasis konsol yang mampu mengelola data produk secara efisien.
2. Mengimplementasikan operasi CRUD (Create, Read, Update, Delete) pada data produk elektronik.

3. Menerapkan algoritma pengurutan secara manual, meliputi Bubble Sort, Selection Sort, dan Insertion Sort, untuk mengelola tampilan data produk.
4. Menerapkan algoritma pencarian secara manual, meliputi Linear Search dan Binary Search, untuk memudahkan pencarian data berdasarkan nama, kategori, maupun ID produk.
5. Menggunakan mekanisme *soft delete* agar riwayat data produk yang dihapus tetap tersimpan dalam log sistem.
6. Membuktikan bahwa konsep dasar algoritma dan struktur data dapat menjadi fondasi yang kuat dalam pengembangan aplikasi yang fungsional dan terstruktur.

BAB II

ANALISIS KEBUTUHAN

2.1 Analisis Kebutuhan Sistem

Sistem Manajemen Stok Toko Elektronik merupakan program berbasis Java yang dibuat untuk membantu pengelolaan data produk elektronik secara sederhana menggunakan struktur data array tanpa library collection. Sistem ini mampu melakukan pengolahan data produk mulai dari penambahan, pencarian, pengurutan, penghapusan, hingga menampilkan statistik data barang.

Program dibuat menggunakan beberapa algoritma dasar seperti searching, sorting, traversal, counting, serta file handling.

2.2 Kebutuhan Fungsional

1. Tambah Produk
2. Tampilkan Semua Produk
3. Edit Produk
4. Hapus Produk (Soft Delete)
5. Log Penghapusan
6. Cari Nama Produk (Linear Search)
7. Cari ID Produk (Binary Search)
8. Cari Kategori Produk
9. Sorting ID (Bubble Sort)
10. Sorting Nama (Selection Sort)
11. Sorting Stok (Insertion Sort)
12. Statistik Produk
13. Load Data TXT

2.3 Struktur Data yang Digunakan

Sistem menggunakan struktur data array manual tanpa menggunakan library collection seperti ArrayList atau LinkedList.

Struktur data utama:

1. int[] id
2. String[] nama
3. String[] kategori
4. int[] stok

5. double[] harga
6. boolean[] aktif

2.4 Algoritma yang Digunakan

1. **Linear Search:** digunakan untuk mencari data berdasarkan nama dan kategori.
2. **Binary Search:** digunakan untuk mencari ID produk setelah data diurutkan.
3. **Bubble Sort:** digunakan untuk mengurutkan ID produk.
4. **Selection Sort:** digunakan untuk mengurutkan nama produk secara alfabetikal.
5. **Insertion Sort:** digunakan untuk mengurutkan stok produk dari yang terbesar.
6. **Traversal Array:** digunakan untuk menampilkan seluruh data produk
7. **Counting:** digunakan untuk menghitung total stok dan statistik produk.
8. **File Handling:** digunakan untuk membaca data dari file TXT.

2.5 Analisis Kebutuhan Non-Fungsional

1. Bahasa Pemrograman : Java
2. IDE : Visual Studio Code
3. Sistem Operasi : Windows
4. Output : Console / Terminal
5. Penyimpanan Data : Array Manual

2.6 Konsep Pemrograman yang Digunakan

Program menerapkan konsep:

1. Array Manual- Searching Algorithm- Sorting Algorithm
2. Traversal Array- Counting- File Handlin

BAB III

IMPLEMENTASI

3.1 Implementasi Kode dan Penjelasan

- Import Library dan Deklarasi Class

```
import java.io.File;
import java.util.Scanner;

public class ManajemenBarangElektronik {

    static Scanner sc = new Scanner(System.in);
```

Penjelasan Kode:

Blok kode ini digunakan untuk mengimpor library yang diperlukan serta mendeklarasikan class utama program. Scanner dipakai untuk menerima input dari pengguna melalui keyboard, sedangkan File digunakan untuk kebutuhan pengolahan file. Class ManajemenBarangElektronik menjadi tempat seluruh program dijalankan, dan objek sc dibuat agar semua method dapat membaca input pengguna.

- Struktur Data dan Variabel Program

```
// struktur data
static final int MAX = 100;

static int[] id = new int[MAX];
static String[] nama = new String[MAX];
static String[] kategori = new String[MAX];
static int[] stok = new int[MAX];
static double[] harga = new double[MAX];
static boolean[] aktif = new boolean[MAX];

static int jumlahData = 0;
static int nextId = 1;

// log soft delete
static String[] logHapus = new String[MAX];
static int jumlahLog = 0;
```


Penjelasan Kode:

Blok ini berfungsi sebagai tempat penyimpanan seluruh data produk elektronik menggunakan array. Setiap array menyimpan informasi tertentu seperti ID, nama produk, kategori, stok, harga, dan status aktif produk. Konstanta MAX menentukan kapasitas maksimal data yaitu 100 produk. Variabel jumlahData digunakan untuk menghitung jumlah produk yang tersimpan, sedangkan nextId dipakai untuk memberikan ID otomatis pada setiap produk baru. Selain itu, terdapat array logHapus dan jumlahLog yang digunakan untuk mencatat riwayat produk yang dihapus menggunakan metode soft delete.

- Method Main dan Menu Program

```
// main menu
Run | Debug
public static void main(String[] args) {
    initDataAwal();

    int pilihan;
    do {
        System.out.println(x: "\n|=====|");
        System.out.println(x: "|    SISTEM MANAJEMEN STOK TOKO ELEKTRONIK    |");
        System.out.println(x: "|=====|");
        System.out.println(x: "| [CRUD] |");
        System.out.println(x: "| 1. Tambah Produk Baru |");
        System.out.println(x: "| 2. Tampilkan Semua Produk |");
        System.out.println(x: "| 3. Edit Produk (berdasarkan ID) |");
        System.out.println(x: "| 4. Hapus Produk (Soft Delete) |");
        System.out.println(x: "| 5. Lihat Log Penghapusan |");
        System.out.println(x: "|=====|");
        System.out.println(x: "| [SEARCHING] |");
        System.out.println(x: "| 6. Cari berdasarkan Nama (Linear Search) |");
        System.out.println(x: "| 7. Cari berdasarkan ID (Binary Search) |");
        System.out.println(x: "| 8. Cari berdasarkan Kategori |");
        System.out.println(x: "|=====|");
        System.out.println(x: "| [SORTING] |");
        System.out.println(x: "| 9. Urutkan ID Ascending (Bubble Sort) |");
        System.out.println(x: "| 10. Urutkan Nama A-Z (Selection Sort) |");
        System.out.println(x: "| 11. Urutkan Stok Terbanyak (Insertion Sort) |");
        System.out.println(x: "|=====|");
        System.out.println(x: "| [STATISTIK] |");
        System.out.println(x: "| 12. Tampilkan Statistik Data |");
        System.out.println(x: "|=====|");
        System.out.println(x: "| [FILE] |");
        System.out.println(x: "| 13. Load Data dari File |");
        System.out.println(x: "| 0. Keluar |");
        System.out.println(x: "|=====|");
```

Penjelasan Kode:

Blok ini merupakan bagian utama program yang pertama kali dijalankan. Method main() memanggil initDataAwal() untuk mengisi data awal

produk agar program langsung memiliki isi. Setelah itu program menampilkan menu utama menggunakan perulangan do-while, sehingga menu akan terus muncul sampai pengguna memilih keluar. Variabel pilihan digunakan untuk menyimpan nomor menu yang dipilih pengguna.

- Input Menu dan Switch Case

```
System.out.print(s: "Pilih menu: ");
pilihan = sc.nextInt();
sc.nextLine();

switch (pilihan) {
    case 1: tambah(); break;
    case 2: tampilkanSemua(); break;
    case 3: edit(); break;
    case 4: hapus(); break;
    case 5: lihatLog(); break;
    case 6: cariNama(); break;
    case 7: cariId(); break;
    case 8: cariKategori(); break;
    case 9: sortById(); break;
    case 10: sortByNama(); break;
    case 11: sortByStok(); break;
    case 12: tampilkanStatistik(); break;
    case 13: loadDariFile(); break;
    case 0: System.out.println(x: "\n Terima kasih! Program selesai."); break;
    default: System.out.println(x: "\n [!] Pilihan tidak valid.");
}
} while (pilihan != 0);
}
```

Penjelasan Kode:

Blok kode ini digunakan untuk membaca pilihan menu dari pengguna dan menjalankan fitur yang sesuai menggunakan switch-case. Setelah pengguna memasukkan nomor menu, program akan memanggil method tertentu seperti tambah(), tampilkanSemua(), edit(), hapus(), searching, sorting, statistik, maupun load file. Struktur ini membuat program lebih rapi karena setiap fitur dipisahkan ke dalam method masing-masing.

- Inisialisasi Data Awal

```
// data awal
static void initDataAwal() {
    String[][] sample = {
        {"Laptop ASUS VivoBook", "Laptop", "45", "7500000"},
        {"Mouse Logitech M235", "Aksesori", "120", "250000"},
        {"Samsung Galaxy A54", "Smartphone", "60", "4200000"},
        {"Keyboard Mechanical RGB", "Aksesori", "35", "850000"},
        {"Monitor LG 24 inch", "Monitor", "20", "2300000"},
        {"Headphone Sony WH-1000", "Audio", "15", "3500000"},
        {"Flash Drive SanDisk 64GB", "Storage", "200", "85000"},
        {"Laptop Lenovo IdeaPad", "Laptop", "30", "6800000"},
        {"Charger USB-C 65W", "Aksesori", "90", "175000"},
        {"SSD Samsung 500GB", "Storage", "55", "950000"},
        {"Printer Epson L3210", "Printer", "25", "2800000"},
        {"iPhone 13 128GB", "Smartphone", "18", "9500000"},
        {"Router TP-Link Archer C6", "Networking", "40", "650000"},
        {"Webcam Logitech C270", "Aksesori", "70", "320000"},
        {"Hard Disk Seagate 1TB", "Storage", "45", "780000"},
        {"Smart TV Samsung 43 Inch", "Elektronik", "12", "5200000"},
        {"Tablet Xiaomi Pad 6", "Tablet", "28", "4900000"},
        {"Speaker JBL Flip 6", "Audio", "22", "2100000"},
        {"Kabel HDMI 2 Meter", "Aksesori", "150", "75000"},
        {"Canon EOS 15000", "Kamera", "10", "7200000"},
        {"Laptop HP Pavilion", "Laptop", "27", "8900000"},
        {"Mouse Razer DeathAdder", "Aksesori", "65", "450000"},
        {"Smartwatch Xiaomi Band 8", "Wearable", "85", "550000"},
        {"Monitor Samsung 27 Inch", "Monitor", "17", "3100000"},
        {"Power Bank 20000mAh", "Aksesori", "95", "300000"},
        {"AirPods Pro 2", "Audio", "20", "4200000"},
        {"SSD Kingston 1TB", "Storage", "33", "1450000"},
        {"Nintendo Switch OLED", "Gaming", "14", "5800000"},
        {"Keyboard Logitech K120", "Aksesori", "110", "180000"},
        {"Drone DJI Mini 3", "Drone", "8", "8900000"},
    };
};
```

Penjelasan Kode:

Blok ini berfungsi untuk mengisi data awal produk elektronik ke dalam program. Data disimpan terlebih dahulu dalam array dua dimensi sample, yang berisi nama produk, kategori, stok, dan harga. Data ini digunakan agar program memiliki contoh produk tanpa perlu input manual dari pengguna saat pertama dijalankan.

- Perulangan Pengisian Data ke Array

```

    for (int i = 0; i < sample.length; i++) {
        id[jumlahData] = nextId++;
        nama[jumlahData] = sample[i][0];
        kategori[jumlahData] = sample[i][1];
        stok[jumlahData] = Integer.parseInt(sample[i][2]);
        harga[jumlahData] = Double.parseDouble(sample[i][3]);
        aktif[jumlahData] = true;
        jumlahData++;
    }
}

```

Penjelasan Kode:

Blok ini menggunakan perulangan for untuk memindahkan data dari array sample ke array utama program. Setiap produk akan diberikan ID otomatis menggunakan nextId, lalu data nama, kategori, stok, harga, dan status aktif dimasukkan ke array masing-masing. Setelah satu data berhasil dimasukkan, jumlahData akan bertambah untuk menandai jumlah produk yang tersimpan.

- Method Tambah Produk

```

// crud tambah
static void tambah() {
    if (jumlahData >= MAX) {
        System.out.println(x: "\n [!] Kapasitas penuh!");
        return;
    }
    System.out.println(x: "\n TAMBAH PRODUK BARU");
    System.out.print(s: " Nama Produk : ");
    String nm = sc.nextLine();
    System.out.print(s: " Kategori : ");
    String kat = sc.nextLine();
    System.out.print(s: " Stok : ");
    int stk = sc.nextInt();
    System.out.print(s: " Harga (Rp) : ");
    double hrg = sc.nextDouble();
    sc.nextLine();

    id[jumlahData] = nextId++;
    nama[jumlahData] = nm;
    kategori[jumlahData] = kat;
    stok[jumlahData] = stk;
    harga[jumlahData] = hrg;
    aktif[jumlahData] = true;
    jumlahData++;

    System.out.println("\n [v] Produk berhasil ditambahkan! ID: " + (nextId - 1));
}

```

Penjelasan Kode:

Blok ini merupakan method CRUD untuk menambahkan produk baru. Program terlebih dahulu memeriksa apakah kapasitas penyimpanan sudah penuh. Jika masih tersedia, pengguna diminta memasukkan nama produk, kategori, stok, dan harga. Setelah input selesai, data disimpan ke array utama, diberikan ID otomatis, lalu jumlahData ditambah. Di akhir proses, program menampilkan pesan bahwa produk berhasil ditambahkan.

- **Tampilkan semua**

```
static void tampilkanSemua() {  
    System.out.println("\n DAFTAR SEMUA PRODUK");  
    printHeader();  
    int count = 0;  
    for (int i = 0; i < jumlahData; i++) {  
        if (aktif[i]) {  
            printRow(i);  
            count++;  
        }  
    }  
    if (count == 0) System.out.println(" (Tidak ada data produk aktif);  
    System.out.println(" Total: " + count + " produk aktif.");  
}
```

Method ini bertugas menampilkan semua produk yang masih aktif. Perulangan berjalan dari index 0 hingga jumlahData, dan setiap produk yang memiliki status aktif[i] = true akan dicetak menggunakan printRow(i). Variabel count digunakan untuk menghitung berapa banyak produk aktif yang ditemukan. Jika tidak ada sama sekali, program menampilkan pesan kosong. Di akhir selalu ditampilkan total jumlah produk aktif.

- **Edit**

```

static void edit() {
    System.out.println("\n EDIT PRODUK");
    System.out.print(" Masukkan ID produk yang akan diedit: ");
    int targetId = sc.nextInt();
    sc.nextLine();

    int idx = cariIndexById(targetId);
    if (idx == -1 || !aktif[idx]) {
        System.out.println("\n [!] Produk ID " + targetId + " tidak ditemukan.");
        return;
    }
}

```

Bagian pertama method edit() ini meminta pengguna memasukkan ID produk yang ingin diubah. Setelah ID didapat, program mencari posisi index-nya menggunakan cariIndexById(). Jika hasilnya -1 (tidak ditemukan) atau produk sudah tidak aktif, proses langsung dihentikan dengan pesan error dan method dikembalikan menggunakan return.

```

System.out.println("\n Data saat ini:");
    printHeader();
    printRow(idx);

    System.out.println("\n Masukkan data baru (Enter = tidak berubah:");

    System.out.print(" Nama [" + nama[idx] + "]: ");
    String nm = sc.nextLine();
    if (!nm.isEmpty()) nama[idx] = nm;

```

```

System.out.print(" Kategori [" + kategori[idx] + "]: ");
String kat = sc.nextLine();
if (!kat.isEmpty()) kategori[idx] = kat;

System.out.print(" Stok [" + stok[idx] + "]: ");
String stokInput = sc.nextLine();
if (!stokInput.isEmpty()) stok[idx] = Integer.parseInt(stokInput);

System.out.print(" Harga [" + harga[idx] + "]: ");
String hargaInput = sc.nextLine();
if (!hargaInput.isEmpty()) harga[idx] = Double.parseDouble(hargaInput);

System.out.println("\n [v] Data produk ID " + targetId + " berhasil
diperbarui.");
}

```

Bagian kedua menampilkan data produk saat ini, lalu meminta input baru satu per satu untuk setiap field. Jika pengguna menekan Enter tanpa mengetik apapun, nilai lama dipertahankan. Jika ada input baru, nilai di array langsung diganti. Khusus untuk stok dan harga, input string dikonversi ke tipe numerik menggunakan `Integer.parseInt()` dan `Double.parseDouble()`.

- **Hapus**

```
static void hapus() {  
    System.out.println("\n HAPUS PRODUK (SOFT DELETE)");  
    System.out.print(" Masukkan ID produk yang akan dihapus: ");  
    int targetId = sc.nextInt();  
    sc.nextLine();  
  
    int idx = cariIndexById(targetId);  
  
    if (idx == -1 || !aktif[idx]) {  
        System.out.println("\n [!] Produk ID " + targetId + " tidak ditemukan atau  
sudah dihapus.");  
        return;  
    }  
}
```

Bagian pertama meminta ID produk yang ingin dihapus, lalu mencari index-nya. Jika tidak ditemukan atau sudah dihapus sebelumnya, proses langsung dihentikan.

```
System.out.println("\n Produk yang akan dihapus:");  
    printHeader();  
    printRow(idx);  
  
    System.out.print("\n Yakin ingin menghapus? (y/n): ");  
    String konfirmasi = sc.nextLine();  
  
    if (konfirmasi.equalsIgnoreCase("y")) {
```



```

logHapus[jumlahLog++] = "[LOG] ID=" + id[idx]
    + " | Nama=" + nama[idx]
    + " | Kategori=" + kategori[idx]
    + " | Stok=" + stok[idx]
    + " | Harga=" + hargaFormatted(idx)
    + " | STATUS: DIHAPUS";
aktif[idx] = false;

System.out.println("\n [v] Produk ID " + targetId + " berhasil dihapus (soft
delete).");
} else {
    System.out.println("\n [!] Penghapusan dibatalkan.");
}
}

```

Bagian kedua menampilkan data produk sebagai konfirmasi visual, lalu meminta persetujuan pengguna. Jika pengguna mengetik y, detail produk dicatat dulu ke array logHapus sebagai riwayat, kemudian aktif[idx] diubah menjadi false. Data tidak benar-benar terhapus dari memori, hanya ditandai tidak aktif — inilah yang disebut **soft delete**. Jika pengguna mengetik selain y, proses dibatalkan.

- **Lihat log**

```

static void lihatLog() {
    System.out.println("\n LOG PENGHAPUSAN PRODUK");
    if (jumlahLog == 0) {
        System.out.println(" (Belum ada produk yang dihapus)");
        return;
    }
    for (int i = 0; i < jumlahLog; i++) {
        System.out.println(" " + (i + 1) + ". " + logHapus[i]);
    }
}

```

Method ini menampilkan riwayat semua produk yang pernah dihapus. Jika jumlahLog masih 0 berarti belum ada produk yang dihapus, sehingga ditampilkan pesan kosong dan method langsung berhenti. Jika sudah ada, program melakukan perulangan dan mencetak setiap entri log dengan nomor urut di depannya.

- **Cari nama**

```
static void cariNama() {  
    System.out.println("\n CARI PRODUK BERDASARKAN NAMA (Linear  
Search)");  
    System.out.print(" Masukkan kata kunci nama: ");  
    String keyword = sc.nextLine().toLowerCase();  
  
    System.out.println("\n Hasil pencarian untuk \"" + keyword + "\"");  
    printHeader();  
    int count = 0;  
    for (int i = 0; i < jumlahData; i++) {  
        if (aktif[i] && nama[i].toLowerCase().contains(keyword)) {  
            printRow(i);  
            count++;  
        }  
    }  
    if (count == 0) System.out.println(" (Tidak ada produk yang cocok)");  
    else System.out.println(" Ditemukan: " + count + " produk.");  
}
```

Method ini mencari produk menggunakan algoritma Linear Search berdasarkan kata kunci nama. Kata kunci dan nama produk keduanya dikonversi ke huruf kecil dengan .toLowerCase() agar pencarian tidak sensitif terhadap kapital. Perulangan mengecek dua kondisi sekaligus: produk harus aktif dan nama produk harus

mengandung kata kunci menggunakan `.contains()`. Setiap produk yang cocok langsung dicetak, dan di akhir ditampilkan total hasil yang ditemukan.

- **Cari id**

```
static void cariId() {  
    System.out.println("\n CARI PRODUK BERDASARKAN ID (Binary  
Search)");  
  
    System.out.print(" Masukkan ID yang dicari: ");  
    int targetId = sc.nextInt();  
    sc.nextLine();  
  
    // Kumpulkan index produk yang aktif  
    int[] idx = new int[jumlahData];  
    int n = 0;  
    for (int i = 0; i < jumlahData; i++) {  
        if (aktif[i]) idx[n++] = i;  
    }  
}
```

Bagian pertama mengumpulkan semua index produk yang aktif ke dalam array sementara `idx`. Hal ini diperlukan karena tidak semua slot di array utama berisi produk aktif akibat soft delete sebelumnya.

```
for (int i = 0; i < n - 1; i++) {  
    for (int j = 0; j < n - i - 1; j++) {  
        if (id[idx[j]] > id[idx[j + 1]]) {  
            int tmp = idx[j];  
            idx[j] = idx[j + 1];  
            idx[j + 1] = tmp;  
        }  
    }  
}
```

Bagian kedua mengurutkan array idx menggunakan **Bubble Sort** berdasarkan nilai ID. Dua elemen yang bersebelahan dibandingkan, dan jika urutannya terbalik maka posisinya ditukar. Proses ini diulang hingga seluruh elemen terurut dari ID terkecil ke terbesar. Pengurutan ini wajib dilakukan terlebih dahulu karena **Binary Search** hanya dapat bekerja pada data yang sudah dalam kondisi terurut.

- Binary Search (cariId)

```
// Binary Search
int kiri = 0;
int kanan = n - 1;
int hasil = -1;
while (kiri <= kanan) {
    int tengah = (kiri + kanan) / 2;
    int idTengah = id[idx[tengah]];
    if (idTengah == targetId) {
        hasil = idx[tengah];
        break;
    } else if (idTengah < targetId) {
        kiri = tengah + 1;
    } else {
        kanan = tengah - 1;
    }
}

if (hasil == -1) {
    System.out.println("\n [!] Produk dengan ID " + targetId + " tidak
ditemukan.");
} else {
    System.out.println("\n Produk ditemukan:");
}
```

```

        printHeader();
        printRow(hasil);
    }
}

```

Penjelasan Kode:

Ini adalah Binary Search. Setiap iterasi ambil posisi tengah, bandingkan dengan **targetId**. Kalau cocok, simpan index aslinya ke **hasil** dan stop. Kalau target lebih besar, geser batas kiri. Kalau lebih kecil, geser batas kanan. Kalau **hasil** masih **-1** berarti tidak ketemu. Kalau ketemu, tampilkan baris datanya.

- Cari berdasarkan kategori (cariKategori) Linear Search

```

// cari berdasarkan kategori (keyword)
static void cariKategori() {
    System.out.println("\n CARI PRODUK BERDASARKAN KATEGORI");
    System.out.print(" Masukkan nama kategori: ");
    String keyword = sc.nextLine().toLowerCase();

    System.out.println("\n Produk dalam kategori \"" + keyword + "\"");
    printHeader();
    int count = 0;
    for (int i = 0; i < jumlahData; i++) {
        if (aktif[i] && kategori[i].toLowerCase().contains(keyword)) {
            printRow(i);
            count++;
        }
    }
    if (count == 0) System.out.println(" (Tidak ada produk dalam kategori tersebut)");
    else System.out.println(" Ditemukan: " + count + " produk.");
}

```

Penjelasan Kode:

Method ini hanya melakukan **linear search** dari index 0 sampai akhir. Setiap produk yang aktif dan nama kategorinya mengandung *keyword* yang dimasukkan user (*case-insensitive* lewat **.toLowerCase()**) akan langsung dicetak. Tidak ada pengurutan sama sekali di sini, cukup satu kali iterasi penuh.

- Urutkan Berdasarkan ID (sortById) Bubble Sort

```
// sorting ID (bubble sort)
static void sortById() {
    System.out.println("\n URUTKAN BERDASARKAN ID ASCENDING
(Bubble Sort)");
    for (int i = 0; i < jumlahData - 1; i++) {
        for (int j = 0; j < jumlahData - i - 1; j++) {
            if (id[j] > id[j + 1]) {
                swap(j, j + 1);
            }
        }
    }
    System.out.println("    [v] Data berhasil diurutkan berdasarkan ID
(Ascending).");
    tampilkanSemua();
}
```

Penjelasan Kode:

Menggunakan bubble sort secara ascending. Dua loop bersarang membandingkan **id[j]** dan **id[j+1]**, kalau yang kiri lebih besar, keduanya ditukar lewat method **swap()**. Setiap iterasi loop luar, elemen terbesar menggelembung ke posisi paling akhir, makanya range loop dalam dikurangi **i** di setiap putaran.

- Urutkan Berdasarkan Nama (sortByNama) Selection Sort

```
static void sortByNama() {
```

```

        System.out.println("\n URUTKAN BERDASARKAN NAMA A-Z (Selection
Sort)");
        for (int i = 0; i < jumlahData - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < jumlahData; j++) {
                if (nama[j].compareToIgnoreCase(nama[minIdx]) < 0) {
                    minIdx = j;
                }
            }
            if (minIdx != i) {
                swap(i, minIdx);
            }
        }
        System.out.println(" [v] Data berhasil diurutkan berdasarkan Nama (A-Z).");
        tampilkanSemua();
    }

```

Penjelasan Kode:

Menggunakan **selection sort**. Di setiap iterasi **i**, cari index elemen dengan nama paling kecil secara alfabetis (pakai **compareToIgnoreCase**) dari posisi **i+1** ke akhir. Kalau index minimum yang ditemukan bukan **i** itu sendiri, baru dilakukan *swap*. Efeknya, sisi kiri array perlahan terisi nama-nama yang sudah terurut A-Z.

- Urutkan Berdasarkan Stok (sortByStok) Insertion Sort

```

static void sortByStok() {
    System.out.println("\nURUTKAN BERDASARKAN STOK TERBANYAK
(Insertion Sort)");
    for (int i = 1; i < jumlahData; i++) {
        // Simpan elemen ke-i sebagai key
        int keyId = id[i];
        String keyNama = nama[i];

```

```

String keyKategori = kategori[i];
int   keyStok    = stok[i];
double keyHarga   = harga[i];
boolean keyAktif  = aktif[i];

int j = i - 1;
// Geser elemen yang stoknya lebih kecil dari key ke kanan
while (j >= 0 && stok[j] < keyStok) {
    id[j + 1]    = id[j];
    nama[j + 1]  = nama[j];
    kategori[j + 1] = kategori[j];
    stok[j + 1]   = stok[j];
    harga[j + 1]  = harga[j];
    aktif[j + 1]  = aktif[j];
    j--;
}
id[j + 1]    = keyId;
nama[j + 1]  = keyNama;
kategori[j + 1] = keyKategori;
stok[j + 1]   = keyStok;
harga[j + 1]  = keyHarga;
aktif[j + 1]  = keyAktif;
}

System.out.println(" [v] Data berhasil diurutkan berdasarkan Stok Terbanyak
(Descending).");
tampilkanSemua();
}

```

Penjelasan Kode:

Menggunakan **insertion sort** secara *descending* (stok terbanyak di atas). Di setiap iterasi **i**, elemen ke-**i** disimpan sebagai **key**. Lalu elemen-elemen di sebelah kiri yang stoknya lebih kecil dari **key** digeser satu posisi ke kanan untuk membuka slot.

Setelah pergeseran selesai, **key** dimasukkan ke posisi yang tepat. Karena data punya banyak *field* (id, nama, kategori, dll.), semua field ikut disimpan dan dipindahkan secara bersamaan, bukan hanya stok nya saja.

- **Inisialisasi Statistik**

```
1 static void tampilkanStatistik() {
2     System.out.println("\n STATISTIK DATA PRODUK");
3     System.out.println(
4         "=====");
5     int totalProduk = 0;
6     int totalStok = 0;
7     int stokTertinggi = -1;
8     int stokTerendah = Integer.MAX_VALUE;
9     double hargaTertinggi = -1;
10    double hargaTerendah = Double.MAX_VALUE;
11    String namaStokTertinggi = "";
12    String namaStokTerendah = "";
13    String namaHargaTertinggi = "";
14    String namaHargaTerendah = "";
15 }
```

Bagian ini berfungsi untuk menginisialisasi variabel yang digunakan dalam proses perhitungan statistik produk. Variabel total Produk dan total Stok digunakan untuk menghitung jumlah produk aktif dan total stok barang. Variabel stok Tertinggi, stok Terendah, harga Tertinggi, dan harga Terendah digunakan untuk mencari nilai maksimum dan minimum. Sedangkan variabel bertipe String digunakan untuk menyimpan nama produk yang memiliki stok atau harga tertentu.

- **Load Data dari File**

```
1 static void loadDariFile() {
2     System.out.println("\n LOAD DATA DARI FILE");
3     System.out.print(" Masukkan nama file (contoh: produk.txt): ");
4     String namaFile = sc.nextLine();
5
6     int berhasil = 0;
7     int gagal = 0;
8 }
```

Method loadDariFile() digunakan untuk membaca data produk dari file teks (.txt) dan memasukkannya ke dalam sistem secara otomatis. Dengan adanya fitur ini, pengguna dapat menambahkan banyak data sekaligus tanpa harus memasukkan data satu per satu melalui menu input.

- **Pencarian Data Berdasarkan ID**

A screenshot of a code editor showing a C++ function named cariIndexById. The function is a static method that takes an integer targetId as a parameter. It uses a for loop to iterate through an array id from index 0 to jumlahData-1. Inside the loop, it checks if id[i] equals targetId. If it does, it returns the index i. If the loop completes without finding the target, it returns -1. The code is written in C++ with standard syntax highlighting.

```
// swap helper
static int cariIndexById(int targetId) {
    for (int i = 0; i < jumlahData; i++) {
        if (id[i] == targetId) return i;
    }
    return -1;
}
```

Method cariIndexById() digunakan untuk mencari posisi suatu produk berdasarkan ID yang dimasukkan oleh pengguna. Hasil pencarian berupa indeks data yang nantinya digunakan dalam proses edit, hapus, maupun pengelolaan data lainnya.

Metode yang digunakan:

- Linear Search

- **Pertukaran Data (Swap)**

```
1 static void swap(int i, int j) {  
2     int tmpId = id[i];  
3     String tmpNama = nama[i];  
4     String tmpKategori = kategori[i];  
5     int tmpStok = stok[i];  
6     double tmpHarga = harga[i];  
7     boolean tmpAktif = aktif[i];  
8  
9     id[i] = id[j];  
10    nama[i] = nama[j];  
11    kategori[i] = kategori[j];  
12    stok[i] = stok[j];  
13    harga[i] = harga[j];  
14    aktif[i] = aktif[j];  
15  
16    id[j] = tmpId;  
17    nama[j] = tmpNama;  
18    kategori[j] = tmpKategori;  
19    stok[j] = tmpStok;  
20    harga[j] = tmpHarga;  
21    aktif[j] = tmpAktif;  
22 }
```

Method swap() digunakan untuk menukar posisi dua data produk yang berada pada indeks berbeda dalam array. Karena data produk disimpan pada beberapa array (id, nama, kategori, stok, harga, dan aktif), maka seluruh data harus ditukar secara bersamaan agar informasi produk tetap sesuai dan tidak tertukar.

- **Format harga produk**

```
static String hargaFormatted(int i) {  
    String s = Long.toString((long) harga[i]);  
    StringBuilder sb = new StringBuilder();  
    int count = 0;  
    for (int k = s.length() - 1; k >= 0; k--) {  
        if (count > 0 && count % 3 == 0) sb.insert(0, '.');  
        sb.insert(0, s.charAt(k));  
        count++;  
    }  
    return "Rp" + sb.toString();  
}
```

Method `hargaFormatted()` digunakan untuk mengubah nilai harga produk menjadi format mata uang Rupiah yang lebih mudah dibaca oleh pengguna. Format ini menambahkan tanda titik (.) sebagai pemisah ribuan dan awalan "Rp" di depan angka.

- **Menampilkan Header Tabel**

```
1 static void printHeader() {
2     System.out.println(
3         "-----");
4     System.out.printf("| %-4s | %-30s | %-15s | %-5s | %-15s |\n",
5         "ID", "Nama Produk", "Kategori", "Stok", "Harga");
6     System.out.println(
7         "-----");
8 }
```

Method `printHeader()` digunakan untuk menampilkan header atau judul kolom pada tabel data produk. Header ini berfungsi sebagai penanda informasi yang akan ditampilkan pada setiap kolom sehingga data lebih mudah dibaca dan dipahami oleh pengguna.

- **Menampilkan Baris Data Produk**

```
1 static void printRow(int i) {
2     System.out.printf("| %-4d | %-30s | %-15s | %-5d | %-15s |\n",
3         id[i],
4         nama[i],
5         kategori[i],
6         stok[i],
7         harga[i]);
8 }
9 }
```

Method `printRow()` digunakan untuk menampilkan satu baris data produk berdasarkan indeks tertentu pada array. Data yang ditampilkan meliputi ID produk, nama produk, kategori, jumlah stok, dan harga produk.

Method ini berfungsi sebagai pelengkap dari `printHeader()`, sehingga seluruh data produk dapat ditampilkan dalam bentuk tabel yang rapi dan mudah dibaca.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Tabel Analisis Kompleksitas Algoritma

Algoritma/Fungsi	Avg Case	Best Case	Worst Case	Keterangan
Linear Search cariByNama() cariByKategori()	$O(n)$	$O(1)$	$O(n)$	Best Case: elemen target ada di indeks pertama. Worst Case: target di akhir atau tidak ditemukan setelah seluruh array ditelusuri.
Binary Search cariById()	$O(\log n)$	$O(1)$	$O(\log n)$	Best Case: target tepat di posisi tengah. Syarat: data harus terurut. Efisien untuk dataset besar karena pencarian membagi ruang separuh tiap iterasi.
Bubble Sort sortById()	$O(n^2)$	$O(n)$	$O(n^2)$	Best Case: data sudah terurut. Worst Case: data terurut terbalik, setiap elemen perlu di-swap secara maksimal di setiap pass.
Selection Sort sortByNama()	$O(n^2)$	$O(n^2)$	$O(n^2)$	Best dan Worst Case sama. Algoritma selalu mencari minimum/maksimum di sisa array. Tidak ada kondisi early-exit, tidak memandang kondisi awal

				data.
--	--	--	--	-------

4.2 Penjelasan Analisis Kompleksitas Algoritma

1. Linear Search

Algoritma ini menelusuri setiap elemen array satu per satu dari indeks pertama hingga terakhir. Kompleksitas rata-rata dan terburuk adalah $O(n)$ karena dalam kondisi terburuk (data tidak ditemukan atau ada di posisi akhir), seluruh n elemen harus diperiksa. Best case $O(1)$ hanya terjadi jika data target berada tepat di indeks pertama.

Cocok digunakan pada atribut seperti nama dan kategori karena data tidak terurut secara natural untuk atribut tersebut,

2. Binary Search

Bekerja dengan membagi rentang pencarian menjadi dua setiap iterasi (*divide and conquer*). Kompleksitas $O(\log n)$ menjadikannya jauh lebih efisien dibanding Linear Search pada dataset besar, misalnya, pada 1.000 data, Binary Search maksimal butuh ~10 perbandingan vs. Linear Search hingga 1.000.

Syarat: data harus terurut berdasarkan id. Jika urutan tidak dijamin, hasil pencarian bisa salah atau tidak ditemukan meski data ada.

3. Bubble Sort

Membandingkan dua elemen berdekatan secara berulang dan menukarnya jika tidak sesuai urutan. Worst case $O(n^2)$ terjadi ketika data terurut terbalik. Best case $O(n)$ hanya dapat dicapai jika diimplementasikan dengan *early termination flag*, jika dalam satu pass tidak ada swap, proses dihentikan.

4. Selection Sort

Mencari elemen terkecil atau terbesar dari sisa array lalu menempatkannya di posisi yang benar. Berbeda dari Bubble Sort, **tidak ada kondisi early-exit**, algoritma selalu menjalankan $n(n-1)/2$ perbandingan tanpa memandang kondisi awal data. Oleh karena itu best case dan worst case-nya identik $O(n^2)$.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan analisis yang telah dilakukan, Kelompok 4 telah berhasil membangun Sistem Manajemen Stok Toko Elektronik berbasis konsol menggunakan bahasa pemrograman Java sebagai bentuk implementasi nyata dari konsep algoritma dan struktur data. Sistem ini mampu mengelola data produk secara efisien melalui operasi CRUD (Create, Read, Update, Delete) , di mana struktur data *array* manual tanpa bantuan *library collection* terbukti efektif menjadi fondasi penyimpanan data yang terstruktur untuk kapasitas maksimal 100 produk. Melalui sistem ini, seluruh *field* data seperti ID, nama, kategori, stok, harga, dan status aktif dapat dikelola secara sinkron. Dalam hal pencarian data, metode *Linear Search* berhasil diterapkan untuk menemukan produk berdasarkan nama dan kategori yang tidak terurut secara natural , sedangkan *Binary Search* terbukti memberikan efisiensi pencarian yang optimal dengan kompleksitas $O(\log n)$ untuk pencarian ID produk yang telah diurutkan terlebih dahulu.

Selain itu, tiga algoritma pengurutan manual juga telah berhasil diintegrasikan dengan karakteristiknya masing-masing, meliputi *Bubble Sort* untuk mengurutkan ID produk secara *ascending* , *Selection Sort* untuk mengurutkan nama produk secara alfabetis , dan *Insertion Sort* untuk mengurutkan stok produk dari yang terbesar secara *descending* , di mana ketiganya memiliki kompleksitas waktu terburuk sebesar $O(n^2)$. Terakhir, keamanan riwayat data pada toko berhasil ditingkatkan melalui mekanisme *soft delete* memanfaatkan array *logHapus* dan variabel *jumlahLog*. Mekanisme ini memastikan bahwa data produk yang dihapus tidak hilang secara permanen melainkan tetap tersimpan di dalam log sistem, sehingga keakuratan informasi operasional toko tetap terjaga dengan baik.